TabTank — Updated Build Plan (v3)

Turn your new tab into an aquarium. Draw your own fish and watch them swim.

Complete Build Plan — Chrome Extension

Table of Contents

- What You Already Have
- What Needs to Change
- Phase 1 — Skeleton (Get Something On Screen)
- Phase 2 — Drawing Tool (painter.js)
- Phase 3 — Fish Spawning & Swimming (fish.js)
- Phase 4 — Ocean Backgrounds & Mode System
- Phase 5 — Bait Feature
- Phase 6 — Persistence (storage.js)
- Phase 7 — Bubbles (bubbles.js) — Optional Polish
- Phase 8 — Custom UI & Button Art
- Layer Stack & Z-Index Order
- Milestone Checklist

What You Already Have

manifest.json (Complete)

Manifest V3 Chrome extension that overrides the new tab page with `newtab.html`. Permissions for `storage` and `unlimitedStorage` are already declared. Icons referenced at 16, 48, and 128px sizes.

```
{
  "manifest_version": 3,
  "name": "TabTank",
  "version": "1.0.0",
  "description": "Turn your new tab into an aquarium. Draw your own fish and wat
  "chrome_url_overrides": {
    "newtab": "newtab.html"
  },
  "permissions": ["storage", "unlimitedStorage"],
  "icons": {
    "16": "icons/icon16.png",
    "48": "icons/icon48.png",
    "128": "icons/icon128.png"
  }
}
```

Git Repository Initialized

newtab.html (Needs Updating)

Current version has a small draw panel in the bottom-right corner. This needs to be replaced with the new fullscreen drawing overlay design (see "What Needs to Change" below).

css/style.css (Needs Updating)

Basic layout rules exist for body, ocean, fish canvas, and bait bucket. The draw panel rules need to be replaced with the new overlay and floating button styles.

What Needs to Change

The drawing experience is being redesigned. Instead of a small draw panel always visible in the bottom-right corner, the new design uses:

1. **A floating "Draw" button** — a single button visible on the aquarium screen (like a `+` or pencil icon)
2. **A fullscreen drawing overlay** — clicking the button opens a fullscreen view where the entire page becomes the drawing canvas with a toolbar

This is a better UX because: - More drawing space — you're not cramped into a 300×200 box - Cleaner aquarium view — no panel cluttering the screen - Room for new features — fish naming, larger brush preview, etc.

Changes to newtab.html

Remove: - The `<div id="drawPanel">` and everything inside it (the small canvas, controls div, all buttons)

Add: - A `<button id="drawBtn">` — floating button visible on the main aquarium screen - A `<div id="drawOverlay">` — hidden fullscreen container with: - `<canvas id="drawCanvas">` — now much larger (fills most of the overlay) - `<div id="toolbar">` — a bar (top or side) containing: - `<input type="color" id="colorPicker">` — color picker - `<input type="range" id="brushSlider">` — brush size slider - `<button id="eraserBtn">` — eraser - `<button id="clearBtn">` — clear canvas - `<input type="text" id="fishName" placeholder="Name your fish">` — fish name input - `<button id="spawnBtn">` — spawn (saves the fish and closes the overlay) - `<button id="closeBtn">` — close without spawning (discard drawing)

Changes to css/style.css

Remove: - `#drawPanel` rules - `#controls` rules

Add: - `#drawBtn` — floating button (`position: fixed` , round, z-index 10, placed wherever you want) - `#drawOverlay` — fullscreen overlay (`position: fixed` , full width/height, z-index 20, `display: none` by default, dark/semi-transparent background) - `#drawCanvas` — large canvas inside the overlay (white background, centered, takes up most of the screen) - `#toolbar` — bar with flex row layout for the controls - `#closeBtn` — close/cancel button

Keep unchanged: - `body` , `#ocean` , `#fishCanvas` , `#baitBucket` — all stay the same

Changes to painter.js

Add: - Click handler on `#drawBtn` that shows the overlay (`drawOverlay.style.display = 'flex'`) - Click handler on `#closeBtn` that hides the overlay (`drawOverlay.style.display =`

`'none'`) and clears the canvas - Click handler on `#spawnBtn` that exports the drawing, creates the fish (with the name from the text input), hides the overlay, and clears the canvas - Fish name gets stored in the fish object as a `name` property

Keep unchanged: - Drawing logic (mousedown, mousemove, mouseup, mouseleave) — same events, just on a bigger canvas - Color picker, brush slider, eraser, clear — same logic, just targeting the new element IDs

---

Phase 1 — Skeleton (Get Something On Screen)

Step 1.1: File Structure

```
TabTank/
├── manifest.json              ✅ exists
├── newtab.html                ⚠️ needs updating
├── css/
│   └── style.css              ⚠️ needs updating
├── js/
│   ├── painter.js
│   ├── fish.js
│   ├── bubbles.js
│   └── storage.js
├── assets/
│   ├── backgrounds/
│   │   ├── ocean/             (standard ocean art)
│   │   └── pixel/             (pixel art backgrounds, 320×180)
│   └── ui/                    (custom button art)
└── icons/
    ├── icon16.png             ✅ exists
    ├── icon48.png             ✅ exists
    └── icon128.png            ✅ exists
```

Step 1.2: newtab.html

The HTML structure has three layers for the aquarium view, plus a hidden drawing overlay:

Aquarium view (always visible): - `<div id="ocean">` — background container for your art - `<canvas id="fishCanvas">` — fullscreen transparent fish swimming surface - `<div`

`id="baitBucket">` — bait feature element - `<button id="drawBtn">` — the floating button that opens the drawing overlay (e.g. a `+` icon or pencil icon)

Drawing overlay (hidden by default, shown when drawBtn is clicked): - `<div id="drawOverlay">` — fullscreen container, hidden with `display: none` - `<div id="toolbar">` — horizontal bar at the top with all controls: - `<input type="text" id="fishName" placeholder="Name your fish">` — name input - `<input type="color" id="colorPicker" value="#000000">` — color picker - `<input type="range" id="brushSlider" min="1" max="20" value="4">` — brush size - `<button id="eraserBtn">Eraser</button>` - `<button id="clearBtn">Clear</button>` - `<button id="spawnBtn">Spawn</button>` — saves the fish and closes overlay - `<button id="closeBtn">×</button>` — close without saving - `<canvas id="drawCanvas" width="800" height="500">` — the large drawing surface

Script tags (at the bottom, before `</body>`): - `storage.js`, `fish.js`, `painter.js`, `bubbles.js` — in that order

Step 1.3: style.css

Core layout rules:

Aquarium view elements (unchanged):

SELECTOR	KEY PROPERTIES
`body`	`margin: 0; overflow: hidden; width: 100vw; height: 100vh; background: #0a1628;`
`#ocean`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 0;`
`#fishCanvas`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 1; pointer-events: none;`
`#baitBucket`	`position: fixed; z-index: 5; cursor: pointer;`

New — Floating draw button:

SELECTOR	KEY PROPERTIES
`#drawBtn`	`position: fixed; bottom: 30px; right: 30px; z-index: 10; width: 60px; height: 60px; border-radius: 50%; border: none; background:`

SELECTOR	KEY PROPERTIES
	`rgba(0,0,0,0.6); color: white; font-size: 28px; cursor: pointer;`

This creates a round floating button in the bottom-right corner. You can replace it with a custom Procreate-drawn image later.

New — Drawing overlay:

SELECTOR	KEY PROPERTIES
`#drawOverlay`	`position: fixed; top: 0; left: 0; width: 100%; height: 100%; z-index: 20; display: none; flex-direction: column; align-items: center; background: rgba(0,0,0,0.85);`
`#toolbar`	`display: flex; align-items: center; gap: 12px; padding: 12px; background: rgba(0,0,0,0.7); width: 100%; justify-content: center;`
`#drawCanvas`	`background: white; border-radius: 8px; cursor: crosshair; margin-top: 20px;`
`#fishName`	`padding: 6px 10px; border-radius: 6px; border: none; font-size: 14px;`
`#closeBtn`	`position: absolute; top: 12px; right: 12px; background: none; border: none; color: white; font-size: 24px; cursor: pointer;`

> **Key detail:** `#drawOverlay` uses `display: none` by default — it's completely hidden. When the user clicks the draw button, JavaScript changes it to `display: flex` to show it. When they spawn or close, it goes back to `display: none`.
>
> **Why `z-index: 20`?** The overlay needs to sit above everything — above the ocean (0), fish canvas (1), bait bucket (5), and draw button (10). Setting it to 20 ensures nothing pokes through.

Step 1.4: Test It

1. Go to `chrome://extensions` in your browser

2. Enable **Developer mode** (toggle in the top right)
3. Click **Load unpacked** → select your TabTank folder (or refresh if already loaded)
4. Open a new tab

You should see: a dark blue screen with a round floating button in the bottom-right corner. Clicking the button should do nothing yet (no JavaScript wired up). The drawing overlay is hidden. This proves the basic layout loads.

> **Testing Workflow (use this for every change going forward):** After every code change: go to `chrome://extensions`, click the refresh icon on your extension card, then open a new tab to see the changes.

Phase 2 — Drawing Tool (painter.js)

Step 2.1: Get the Canvas Context

- Grab `#drawCanvas` with `document.getElementById`
- Call `canvas.getContext('2d')` — store the result as `ctx`
- Set initial drawing values:
- `ctx.lineCap = 'round'` — rounds the ends of lines
- `ctx.lineJoin = 'round'` — rounds the corners where lines meet
- `ctx.lineWidth = 4` — default brush thickness
- `ctx.strokeStyle = '#000000'` — default color: black

Step 2.2: Wire Up the Overlay Toggle

Grab the draw button, overlay, and close button with `getElementById`.

Open the overlay:

```
drawBtn.addEventListener('click', function() {
    drawOverlay.style.display = 'flex';
```

```
});
```

Close the overlay (discard):

```
closeBtn.addEventListener('click', function() {
    drawOverlay.style.display = 'none';
    ctx.clearRect(0, 0, canvas.width, canvas.height);  // clear the canvas
    fishName.value = '';  // clear the name input
});
```

Test checkpoint: Reload extension, open new tab. Click the floating button — the overlay should appear with a white canvas and toolbar. Click the × — it should disappear. The aquarium should be visible behind a dark tinted overlay.

Step 2.3: Implement Freehand Drawing

You need one variable: `let isDrawing = false`

Add four event listeners on the drawing canvas:

EVENT	WHAT IT DOES
`mousedown`	Set `isDrawing = true`. Call `ctx.beginPath()`, then `ctx.moveTo(e.offsetX, e.offsetY)`.
`mousemove`	If `isDrawing` is false, return early (do nothing). Otherwise: call `ctx.lineTo(e.offsetX, e.offsetY)`, then `ctx.stroke()`.
`mouseup`	Set `isDrawing = false`.
`mouseleave`	Also set `isDrawing = false` (stops drawing if cursor leaves the canvas).

Test checkpoint: Open the overlay, draw on the white canvas. You should see black lines following your mouse. The canvas is much larger now — enjoy the extra space.

Step 2.4: Color Picker

Add an `input` event listener on your `<input type="color">` that sets `ctx.strokeStyle` to the picker's current value. Store the active color in a variable so the eraser can restore it later.

> **Test checkpoint:** Draw in different colors. The color picker should change the line color.

Step 2.5: Brush Size

Add an `input` event listener on your range slider. On change, set `ctx.lineWidth` to the slider's current value. A range of 1 to 20 works well.

> **Test checkpoint:** Move the slider and draw. Lines should get thicker/thinner.

Step 2.6: Eraser

Two approaches — pick one:

APPROACH	HOW	TRADEOFF
Simple	Eraser button sets `ctx.strokeStyle = '#ffffff'` (draws in white). When user picks a color again, it switches back.	Only works because the draw canvas has a white background.
Better	Eraser button sets `ctx.globalCompositeOperation = 'destination-out'`. This erases to transparent. When user picks a color, set it back to `'source-over'`.	Works with any background color. Better if you ever change the canvas background.

Step 2.7: Clear Button

Click listener that calls `ctx.clearRect(0, 0, canvas.width, canvas.height)`.

If you're using the simple eraser approach (drawing in white), also fill with white after clearing:

```
ctx.fillStyle = 'white';
ctx.fillRect(0, 0, canvas.width, canvas.height);
```

> **Test checkpoint:** Draw something, erase parts, clear, change colors and sizes. The drawing tool should feel complete and responsive. The fullscreen canvas should feel much more natural to draw on than the old small panel.

Phase 3 — Fish Spawning & Swimming (fish.js)

Step 3.1: Set Up the Fish Canvas

- Grab `#fishCanvas` with `getElementById`
- Get its 2D context: `const fishCtx = fishCanvas.getContext('2d')`
- Set the canvas `width` and `height` **attributes** to `window.innerWidth` and `window.innerHeight`
- Add a `resize` event listener on `window` that updates these dimensions when the browser resizes

> **Important: Canvas Resolution vs CSS Size** CSS `width: 100%` stretches the canvas visually, but the canvas internal resolution is set by its `width` and `height` attributes. If you only set CSS size, your fish will be blurry. You must set both the CSS size AND the element attributes to match the window dimensions.

Step 3.2: Create the Fish Array

`const fishArray = []`

Each fish object in this array will have the following properties:

PROPERTY	TYPE	PURPOSE
`image`	Image	The drawn fish as an Image object
`name`	String	The name the user gave the fish (from the text input)

PROPERTY	TYPE	PURPOSE
`x`	Number	Current horizontal position
`y`	Number	Current vertical position
`speedX`	Number	Horizontal velocity (positive = right, negative = left)
`wobbleOffset`	Number	Random offset so fish don't bob in sync
`width`	Number	Rendered width on screen
`height`	Number	Rendered height on screen
`targetX`	Number or null	Bait target X (null = normal swimming)
`targetY`	Number or null	Bait target Y (null = normal swimming)

Step 3.3: Wire Up the Spawn Button

Add a click listener on `#spawnBtn` that does the following, in order:

1. **Get the fish name:** `const name = fishName.value || 'Unnamed Fish'`
2. **Export the drawing:** `const dataURL = drawCanvas.toDataURL('image/png')` — this converts whatever the user drew into a base64-encoded PNG string.
3. **Create an Image object:** `const img = new Image()` then `img.src = dataURL`
4. **Wait for the image to load:** Everything below goes inside `img.onload = () => { ... }`. You must wait because the image isn't usable until it loads, even from a data URL.
5. **Pick a random starting position:**
6. `x` : either `-100` (start offscreen left) or `canvas.width + 100` (start offscreen right)
7. `y` : random between `50` and `canvas.height - 150` (keep fish away from very top and bottom)
8. **Pick random movement values:**
9. `speedX` : random between `0.5` and `2.5`. If starting from the right side, make it negative.
10. `wobbleOffset` : random between `0` and `Math.PI * 2`
11. **Set dimensions:** Scale the drawing down to about 80–120px wide. Maintain aspect ratio based on the draw canvas dimensions.

12. **Push the fish object** into `fishArray` with all the properties listed in Step 3.2 (including the `name`). Set `targetX` and `targetY` to `null`.
13. **Close the overlay:** Hide `drawOverlay`, clear the draw canvas, clear the name input. The user is returned to the aquarium and their fish starts swimming.

> **Test checkpoint:** Draw a fish, give it a name, click Spawn. The overlay should close and the fish should appear swimming. Check the browser console (F12 → Console tab) — there should be no errors.

Step 3.4: Build the Animation Loop

Create a function called `animate` that will run ~60 times per second. This is the core of the entire project.

Every frame, the function does:

1. **Clear the entire fish canvas:** `fishCtx.clearRect(0, 0, fishCanvas.width, fishCanvas.height)`
2. **Loop through** `fishArray` and for each fish, update its position and draw it:

Position update (normal swimming): `javascript fish.x += fish.speedX fish.y += Math.sin(Date.now() * 0.002 + fish.wobbleOffset) * 0.5` The first line moves the fish horizontally at constant speed. The second line uses `Math.sin()` to create a smooth wave — the `wobbleOffset` makes each fish bob at a different phase, the `0.002` controls how fast the wave oscillates, and the `0.5` controls the amplitude (how far up/down it bobs).

Edge detection: `javascript if (fish.x > fishCanvas.width + fish.width) fish.speedX *= -1; if (fish.x < -fish.width) fish.speedX *= -1;` When the fish moves fully offscreen on either side, reverse its direction.

Draw the fish: (see Step 3.5 for the flip logic)

1. **Request the next frame:** `requestAnimationFrame(animate)` at the end of the function.

Kick off the loop by calling `animate()` once when the page loads.

> **What is `requestAnimationFrame`?** This is the browser's built-in "call this function on the next screen refresh" method. It's smoother than `setInterval`, runs at the display's refresh

> rate (usually 60fps), and automatically pauses when the tab isn't visible (saves CPU).

Step 3.5: The Flip Logic in Detail

This is the trickiest part of the animation. When a fish swims left, the image needs to be mirrored horizontally so it faces the direction it's traveling.

When going left (`fish.speedX < 0`):

```
fishCtx.save();
fishCtx.translate(fish.x + fish.width, fish.y);   // move origin to fish's right
fishCtx.scale(-1, 1);                              // flip horizontally
fishCtx.drawImage(fish.image, 0, 0, fish.width, fish.height);
fishCtx.restore();
```

When going right (`fish.speedX > 0`):

```
fishCtx.drawImage(fish.image, fish.x, fish.y, fish.width, fish.height);
```

`ctx.save()` and `ctx.restore()` ensure the transform (translate + scale) only affects this one fish and doesn't corrupt drawing for the next fish in the loop.

Step 3.6: Display Fish Names (Optional)

After drawing each fish in the animation loop, you can draw its name below it:

```
fishCtx.fillStyle = 'white';
fishCtx.font = '12px Arial';
fishCtx.textAlign = 'center';
fishCtx.fillText(fish.name, fish.x + fish.width / 2, fish.y + fish.height + 15);
```

This renders the name centered below the fish. You could also only show names on hover, or toggle them with a button.

> **Major test checkpoint:** Draw a fish, name it, click Spawn. You should see it swimming across the screen with its name, bobbing gently up and down, and flipping when it hits the edge. Spawn multiple fish — they should all move independently with different speeds and wobble patterns. **If this works, the hard part is done.**

Phase 4 — Ocean Backgrounds & Mode System

This phase covers both your background art and the mode switching system.

Step 4.1: Create Your Art

Open Procreate, Figma, or whatever design tool you prefer.

You need art for at least two modes:

Standard Ocean Mode

Resolution: 1920×1080 or larger, exported as PNG or JPEG.

For animated depth, split into 2–3 layers and export each separately:

LAYER FILE	CONTENT	ANIMATION
`bg-deep.png`	The far background — deep water, gradient, distant elements	None (stays still)
`bg-foreground.png`	Close-up elements — coral, sand floor, seaweed, rocks	Slow horizontal sway
`bg-light.png`	Light rays / caustics (export with transparency as PNG)	Opacity pulse

Save into `assets/backgrounds/ocean/`.

Pixel Art Mode

Resolution: **320×180** — draw at this tiny resolution intentionally. The browser will scale it up to fullscreen with hard pixel edges, creating the chunky pixel art look.

LAYER FILE	CONTENT	ANIMATION
`pixel-bg.png`	Full pixel art ocean scene at 320×180	None or subtle sway

Save into `assets/backgrounds/pixel/`.

> **Tip:** Even 2 layers (background + foreground) with a slight parallax sway makes a big difference. The contrast between a static layer and a moving layer creates a convincing sense of depth with almost zero effort.

Step 4.2: Add a Mode Toggle

Add a mode toggle button to your HTML. This could be near the floating draw button, or as its own floating element:

```
<button id="modeToggle">Switch Mode</button>
```

Step 4.3: Add Image Layers to HTML

Inside `<div id="ocean">`, add `<img>` tags for each layer. Give them classes like `ocean-layer-deep`, `ocean-layer-fg`, `ocean-layer-light`.

You'll have image tags for **each mode's backgrounds** — the mode system will show/hide the right set.

Step 4.4: Style the Layers

Each image layer needs these CSS properties:

```
.ocean-layer {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    object-fit: cover;    /* fills the screen without distortion */
}
```

`object-fit: cover` is critical — it ensures the image fills the entire viewport regardless of screen size, cropping edges if needed rather than leaving gaps.

Step 4.5: Pixel Mode CSS

The key to pixel art mode is one CSS property: `image-rendering: pixelated`.

```
body.pixel-mode #ocean img {
    image-rendering: pixelated;
}

body.pixel-mode #fishCanvas {
    image-rendering: pixelated;
}

body.pixel-mode #drawCanvas {
    image-rendering: pixelated;
}
```

When pixel mode is active, everything renders with hard pixel edges instead of smooth scaling. Your 320×180 backgrounds will look intentionally blocky and retro when blown up to fullscreen.

Optional — Pixel draw canvas: In pixel mode, you could also shrink the draw canvas resolution so drawings come out pixelated. JavaScript would swap the canvas dimensions when the mode changes.

Step 4.6: Mode Switching Logic (JavaScript)

The mode toggle button click does: 1. Toggle a class on `<body>` (e.g. `"pixel-mode"`) 2. Show/hide the correct background images 3. Optionally adjust the draw canvas resolution 4. Save the current mode to Chrome storage so it persists

The CSS handles the visual differences via the body class — when `body` has class `pixel-mode`, the pixel CSS rules kick in. When it doesn't, the standard rules apply. JavaScript just flips the class.

Step 4.7: Animate with CSS Keyframes

Sway animation (for the foreground layer):

```
@keyframes sway {
    0%   { transform: translateX(-5px); }
    100% { transform: translateX(5px); }
}

.ocean-layer-fg {
```

```
    animation: sway 6s ease-in-out infinite alternate;
}
```

Light pulse animation (for the light rays layer):

```
@keyframes lightPulse {
    0%   { opacity: 0.25; }
    100% { opacity: 0.5; }
}

.ocean-layer-light {
    animation: lightPulse 4s ease-in-out infinite alternate;
}
```

The deep background layer gets **no animation** — it stays completely still. This contrast between static and moving layers is what creates the illusion of depth.

> **Test checkpoint:** Reload, open new tab. Your art should fill the screen with subtle animation. Fish should swim on top of it on the transparent canvas layer. Toggle between modes — backgrounds should switch and pixel mode should render with hard pixel edges.

Phase 5 — Bait Feature

Step 5.1: Bait Mode Toggle

You need a variable: `let baitMode = false`

Add a click listener on `#baitBucket` that does:

1. Set `baitMode = true`
2. Change the cursor on `#fishCanvas` to `crosshair`
3. Set `fishCanvas.style.pointerEvents = 'auto'` — this is critical. Earlier you set `pointer-events: none` so clicks pass through the canvas. Now you need the canvas to capture clicks.

Step 5.2: Ocean Canvas Click Handler

Add a `click` event listener on `#fishCanvas`. The handler does:

1. **Guard clause:** If `baitMode` is false, return immediately.
2. **Get click coordinates:** `const mouseX = e.clientX`, `const mouseY = e.clientY`
3. **Find the closest fish:**
4. Set `let closestFish = null` and `let closestDist = Infinity`
5. Loop through every fish in `fishArray`. For each:
   - Calculate the fish's center: `centerX = fish.x + fish.width / 2`, `centerY = fish.y + fish.height / 2`
   - Calculate distance: `Math.sqrt((centerX - mouseX) ** 2 + (centerY - mouseY) ** 2)`
   - If this distance is less than `closestDist`, update `closestFish` and `closestDist`
6. If no fish exist (array is empty), just reset bait mode and return
7. **Set the target on the closest fish:** `closestFish.targetX = mouseX`, `closestFish.targetY = mouseY`
8. **Reset bait mode:**
9. `baitMode = false`
10. Restore cursor to default
11. Set `fishCanvas.style.pointerEvents = 'none'` again

Step 5.3: Update the Animation Loop

In your `animate` function, modify the fish update logic. For each fish, **before** the normal swimming code, add a check:

If `fish.targetX !== null` (fish is chasing bait):

1. **Calculate angle to target:** `const angle = Math.atan2(fish.targetY - fish.y, fish.targetX - fish.x)`
2. **Set rush speed:** `const rushSpeed = 6` (much faster than the normal 0.5–2.5)
3. **Move toward target:** `javascript fish.x += Math.cos(angle) * rushSpeed; fish.y += Math.sin(angle) * rushSpeed;`
4. **Flip the fish image:** Base the direction on the target, not `speedX`. If `fish.targetX < fish.x`, the fish should face left. Otherwise face right.
5. **Check if the fish arrived:** Calculate distance to target. If distance is less than `15` pixels:
6. Set `fish.targetX = null` and `fish.targetY = null`

7. Update `fish.speedX` to continue in the direction it was rushing.

Else (no target): Normal swimming logic (the wobble + drift from Step 3.4).

> **Test checkpoint:** Spawn a few fish, click the bait bucket, click somewhere on the ocean. The nearest fish should dart quickly toward that spot, then resume normal swimming when it arrives.

Optional polish: - **One-click flow:** Right-click anywhere drops bait (skip the bucket) - **All nearby fish react:** Find all fish within a radius, not just the closest - **Bait visual:** Draw a shrinking circle at the click point - **Eating animation:** Fish briefly scales up ~10% when it reaches the bait

---

Phase 6 — Persistence (storage.js)

Step 6.1: Save Function

Create a function called `saveFish()` that:

1. **Maps `fishArray` to serializable objects:** `javascript { dataURL: fish.image.src, // the base64 PNG string name: fish.name, // the fish's name x: fish.x, y: fish.y, speedX: fish.speedX, wobbleOffset: fish.wobbleOffset, width: fish.width, height: fish.height }`
2. **Save to Chrome storage:** `chrome.storage.local.set({ fish: serializedArray })`

Call `saveFish()` at the end of your Spawn button handler (after pushing to the array).

Step 6.2: Load Function

Create a function called `loadFish()` that:

1. Calls `chrome.storage.local.get('fish', (result) => { ... })`
2. If `result.fish` exists and is an array, loops through each saved fish object:
3. Create `const img = new Image()`
4. Set `img.src = saved.dataURL`

5. Inside `img.onload = () => { ... }`, push a full fish object into `fishArray` using the saved properties (including `name`) and the loaded image

Call `loadFish()` when the page loads, **before** starting the animation loop.

> **Why `img.onload`?** Even though the image source is a base64 data URL (not a network request), the browser still needs a moment to decode it. The `onload` callback ensures you don't try to draw the image before it's ready.

Step 6.3: Save Mode Preference

Save the current mode (standard vs pixel) so it persists:

```
chrome.storage.local.set({ mode: currentMode })
```

Load it on startup and apply the correct body class before showing anything.

Step 6.4: Delete a Fish (Optional but Recommended)

METHOD	IMPLEMENTATION	COMPLEXITY
"Clear all fish" button	Empty `fishArray`, call `chrome.storage.local.remove('fish')`	Easy — do this for v1
Right-click a fish	On right-click, find nearest fish, remove from array, save	Medium
Drag to trash icon	Much more complex, skip for v1	Hard

At minimum, add a "Clear all fish" button somewhere accessible.

> **Test checkpoint:** Spawn fish, close the tab, open a new tab. Your fish should reappear with their names and continue swimming. Switch modes, close tab, reopen — mode should persist too.

Phase 7 — Bubbles (bubbles.js) — Optional Polish

Step 7.1: Bubble Array

Create an array of bubble objects, each with:

PROPERTY	VALUE
`x`	Random position across the screen width
`y`	Start at the bottom of the screen
`radius`	Random between 2–6px
`speedY`	Random between 0.3–1.0 (how fast it rises)
`drift`	Random slight horizontal wobble value

Spawn 20–30 bubbles on page load.

Step 7.2: Animate Bubbles

Three approaches — pick one:

APPROACH	HOW	PROS/CONS
Add to fish canvas loop	After drawing fish, draw translucent white circles with `fishCtx.arc()`	Simple, no extra elements.
Separate canvas	A third canvas layered between ocean and fish	Clean separation.
CSS-animated divs	Small round `<div>` s animated upward with CSS `@keyframes`	Easiest. No canvas needed. Recommended.

Phase 8 — Custom UI & Button Art

Step 8.1: Draw Your Buttons in Procreate

Design custom art for: - The floating draw button (`#drawBtn`) - Toolbar buttons (eraser, clear, spawn, close) - Bait bucket - Mode toggle

Export each as PNG with transparency. Save into `assets/ui/` .

For pixel mode, draw a second set at low resolution and save separately.

Step 8.2: Apply Custom Button Art with CSS

Strip default button appearance and use your art:

```
#drawBtn {
    border: none;
    background: url('../assets/ui/draw-btn.png') center/contain no-repeat;
}
```

For pixel mode variants:

```
body.pixel-mode #drawBtn {
    background-image: url('../assets/ui/pixel/draw-btn.png');
    image-rendering: pixelated;
}
```

Step 8.3: Hover & Active States

```
#drawBtn:hover {
    transform: scale(1.1);
}

#drawBtn:active {
    transform: scale(0.95);
}
```

> **Test checkpoint:** All buttons display your custom art. Hover and click feel responsive. Pixel mode swaps to pixel art buttons.

Layer Stack & Z-Index Order

```
┌──────────────────────────────────────────────┐
│ z-index: 0   Ocean background images (your art)│
│ z-index: 1   Fish canvas (transparent,fullscreen)│
│ z-index: 2   Bubbles (if using divs/extra canvas)│
│ z-index: 5   Bait bucket                      │
│ z-index: 10  Floating draw button              │
│ z-index: 20  Drawing overlay (when open)       │
└──────────────────────────────────────────────┘
```

Aquarium view (normal):

```
┌──────────────────────────────────────────────┐
│  bg-deep.png  (static)                        │
│    ↕ bg-foreground.png (slow sway ~5px)       │
│      ✨ bg-light.png (opacity pulse)          │
│                                               │
│        🐟 "Nemo" swimming on canvas           │
│        🐟 "Bubbles" swimming on canvas        │
│        🫧 procedural bubbles floating up      │
│                                               │
│  🪣 bait bucket                          [🎨] │
│                              (draw button) ↗  │
└──────────────────────────────────────────────┘
```

Drawing overlay (when open):

```
┌──────────────────────────────────────────────┐
│ [Name: ___] [🎨] [▬▬] [Eraser] [Clear] [Spawn] [×] │
│                                               │
│   ┌──────────────────────────────────────┐   │
│   │                                      │   │
│   │                                      │   │
│   │         (large white drawing canvas) │   │
```

```
│   │         draw your fish here              │   │
│   │                                          │   │
│   │                                          │   │
│   └──────────────────────────────────────────┘   │
│                                                  │
└──────────────────────────────────────────────────┘
```

Milestone Checklist

Work through these in order. Each one is independently testable.

#	MILESTONE	HOW TO VERIFY
1	Extension loads, dark screen + floating draw button	New tab shows dark blue background with round button in bottom-right
2	Draw button opens fullscreen overlay, close button closes it	Click button → overlay appears. Click ✕ → overlay disappears.
3	Can draw on the large canvas with colors/sizes/eraser/clear	Freehand drawing works in the overlay with all controls
4	Spawn creates a named swimming fish and closes the overlay	Draw, name, spawn → overlay closes, fish swims on screen with name
5	Multiple fish swim independently	Spawn 5+ fish, all have different speeds and wobble at different phases
6	Ocean art shows as background with CSS animation	Your hand-drawn art fills the screen with subtle sway and light pulse
7	Mode toggle switches between standard and pixel art	Backgrounds swap, pixel mode renders with hard pixel edges
8	Bait click makes nearest fish dart to cursor position	Fish rushes to click point then smoothly resumes normal swimming

#	MILESTONE	HOW TO VERIFY
9	Fish, names, and mode persist across new tabs	Close tab, open new one — fish, names, and mode are all preserved
10	Bubbles float upward	Visual polish — translucent circles rise from bottom of screen
11	Custom button art replaces browser defaults	All buttons display your Procreate art with pixel variants

Quick Reference — Key APIs You'll Use

Canvas 2D Context

METHOD / PROPERTY	WHAT IT DOES
`getContext('2d')`	Gets the 2D drawing context from a canvas element
`beginPath()`	Starts a new drawing path
`moveTo(x, y)`	Moves the pen to a position without drawing
`lineTo(x, y)`	Draws a line from current position to (x, y)
`stroke()`	Renders the current path as a visible line
`clearRect(x, y, w, h)`	Erases a rectangular area
`drawImage(img, x, y, w, h)`	Draws an image onto the canvas
`save()` / `restore()`	Saves/restores the current transform state
`translate(x, y)`	Moves the canvas origin point
`scale(x, y)`	Scales drawing. `scale(-1, 1)` flips horizontally

METHOD / PROPERTY	WHAT IT DOES
`toDataURL('image/png')`	Exports canvas content as a base64 PNG string
`strokeStyle`	Sets the line/stroke color
`lineWidth`	Sets the line thickness in pixels
`lineCap`	Sets line end shape (`'round'` for smooth ends)
`fillText(text, x, y)`	Draws text on the canvas (for fish names)
`globalCompositeOperation`	Controls how new drawings blend with existing content

DOM & Events

METHOD	WHAT IT DOES
`document.getElementById('id')`	Finds an HTML element by its ID
`element.addEventListener('event', function)`	Runs code when an event happens (click, mousemove, etc.)
`element.style.display = 'flex'`	Shows a hidden element
`element.style.display = 'none'`	Hides an element
`element.classList.toggle('class')`	Adds/removes a CSS class (used for mode switching)

Math Functions for Animation

FUNCTION	WHAT IT DOES
`Math.sin(value)`	Returns -1 to 1 in a wave pattern. Used for swimming wobble.
`Math.atan2(dy, dx)`	Angle from one point to another. Used for bait chasing.

FUNCTION	WHAT IT DOES
`Math.cos(angle)`	X-component of movement in a given direction
`Math.sin(angle)`	Y-component of movement in a given direction
`Math.sqrt(x**2 + y**2)`	Distance between two points
`Math.random()`	Random number between 0 and 1

Chrome Extension APIs

API	WHAT IT DOES
`chrome.storage.local.set({ key: value })`	Saves data persistently
`chrome.storage.local.get('key', callback)`	Retrieves saved data
`chrome.storage.local.remove('key')`	Deletes saved data

Browser Animation

API	WHAT IT DOES
`requestAnimationFrame(callback)`	Runs a function on next screen refresh (~60fps)
`Date.now()`	Current timestamp in milliseconds. Input for `Math.sin()` for time-based animation.

CSS Pixel Art Mode

PROPERTY	WHAT IT DOES
`image-rendering: pixelated`	Scales images with hard pixel edges instead of smooth blurring